▶ 작업 보고서: 게시글 페이징 Step 3 - UI 제어 및 유효성 검사

- **작업 일시**: 2026-03-23
- **진행 단계**: 완료

1. 🌊 전체 작업 흐름 (Workflow)

사용자 친화적인 페이징 시스템을 구축하기 위해 1-기반 페이지 번호를 도입하고, 잘못된 접근에 대한 방어 로직을 설계했습니다.

```
+-------------------------------------------------------+
| [Header Navbar] (기존 레이아웃)                        |
+-------------------------------------------------------+
|                                                       |
|  ### 게시글 목록 (현재 페이지: 1)                      |
|                                                       |
|  +------+-----------------------+------------+        |
|  | 번호 | 제목                  | 상세보기   |        |
|  +------+-----------------------+------------+        |
|  | 20   | 제목 20               | [상세보기] |        |
|  | 19   | 제목 19               | [상세보기] |        |
|  | 18   | 제목 18               | [상세보기] |        |
|  +------+-----------------------+------------+        |
|                                                       |
|      [ Previous ]    [ Next ]                         |
|                                                       |
+-------------------------------------------------------+
| [Footer] (기존 레이아웃)                               |
+-------------------------------------------------------+
```

1. **예외 클래스 정의**: 존재하지 않는 페이지 요청 시 사용할 `Exception404` 생성.
2. **공통 예외 처리**: `GlobalExceptionHandler`에 404 예외 발생 시 자바스크립트 alert와 함께 이전 페이지로 돌아가는 로직 추가.
3. **컨트롤러 보정**: 0 이하의 페이지 요청 시 1페이지로 강제 이동(Redirect) 및 서비스에 0-기반 인덱스(`page - 1`) 전달.
4. **서비스 유효성 검사**: 조회된 데이터가 없을 경우 `Exception404`를 던져 사용자에게 알림.
5. **UI 개선**: Mustache 템플릿에 `Previous`, `Next` 버튼을 추가하여 실제 페이지 이동 기능 구현.

2. 🧩 변경된 모든 코드 포함

1) Exception404.java (커스텀 예외)

```java
package com.example.demo._core.handler.ex;

// 404 Not Found 상황을 정의하는 예외 클래스
```

```java
public class Exception404 extends RuntimeException {
    public Exception404(String message) {
        super(message);
    }
}
```

2) GlobalExceptionHandler.java (예외 핸들러)

```java
// 404 에러 처리: 메시지를 띄우고 브라우저의 뒤로가기 실행
@ResponseBody
@ExceptionHandler(Exception404.class)
public String handleException404(Exception404 e) {
    var body = """
            <script>
                alert("%s");
                history.back();
            </script>
            """.formatted(e.getMessage());
    return body;
}
```

3) BoardController.java (페이지 번호 보정 및 리다이렉트)

```java
@GetMapping("/")
public String list(@RequestParam(defaultValue = "1", name = "page") int page,
Model model) {
    // 1페이지보다 작은 번호를 요청하면 강제로 1페이지 주소로 이동시킴
    if (page < 1) {
        return "redirect:/?page=1";
    }

    // DB는 0번부터 시작하므로 (page - 1)을 전달
    var boardList = boardService.게시글목록보기(page - 1);

    model.addAttribute("boardList", boardList);
    model.addAttribute("page", page);
    model.addAttribute("prevPage", page - 1); // 이전 버튼용 주소값
    model.addAttribute("nextPage", page + 1); // 다음 버튼용 주소값
    return "board/list";
}
```

4) BoardService.java (데이터 존재 여부 검증)

```java
public List<BoardResponse.ListDTO> 게시글목록보기(int page) {
    int limit = 3;
    int offset = page * limit;
```

```
    var boardList = boardRepository.findAll(limit, offset);

    // 조회된 결과가 비어있다면, 잘못된 페이지를 요청한 것으로 간주하고 예외 발생
    if (boardList.isEmpty()) {
        throw new Exception404("더 이상 게시글이 없습니다.");
    }

    return boardList.stream()
            .map(BoardResponse.ListDTO::new)
            .collect(Collectors.toList());
}
```

5) list.mustache (UI 버튼 추가)

```
<ul class="pagination d-flex justify-content-center">
    <!-- 이전/다음 버튼 링크 연결 -->
    <li class="page-item"><a class="page-link" href="/?page=
{{prevPage}}">Previous</a></li>
    <li class="page-item"><a class="page-link" href="/?page={{nextPage}}">Next</a>
</li>
</ul>
```

3. 💡 상세비유 쉬운 예시를 들어서 (Easy Analogy)

"이번 작업은 **서점의 안내원**과 같습니다. 손님이 '0번 서가로 가고 싶어요'라고 하면 안내원은 '저희 서점은 1번 서가부터 시작합니다!'라며 친절하게 1번 서가로 안내(Redirect)해줍니다. 만약 손님이 책이 하나도 없는 '999번 서가'에 가려고 하면 '죄송합니다, 그곳에는 더 이상 책이 없습니다'라고 정중하게 알려주는(Exception404) 것과 같습니다!"

4. 📚 기술 딥다이브 (Technical Deep-dive)

- **Redirect vs Forward**: 사용자가 잘못된 페이지(0 이하)를 요청했을 때, 내부적으로만 바꾸는 것이 아니라 브라우저 주소창 자체를 `?page=1`로 바꿔줌으로써 주소의 정합성을 유지합니다.
- **1-Based vs 0-Based Mapping**: 사용자는 1페이지를 보지만, 개발자(DB)는 0부터 세는 차이를 컨트롤러에서 `page - 1` 연산을 통해 교량 역할을 수행하도록 설계했습니다. 이를 통해 서비스 로직은 순수하게 '필요한 위치'의 데이터만 가져오는 데 집중할 수 있습니다.
- **@ControllerAdvice와 @ExceptionHandler**: 애플리케이션 곳곳에서 발생하는 예외를 한곳에서 가로채어 공통된 응답(자바스크립트 alert 등)을 처리함으로써 코드의 중복을 획기적으로 줄였습니다.